```
      create      app/views/comments/new.html.erb
      create      app/views/comments/_form.html.erb
      invoke    shoulda
      create      test/controllers/comments_controller_test.rb
      invoke    my_helper
      create      app/helpers/comments_helper.rb
      invoke    jbuilder
      create      app/views/comments/index.json.jbuilder
      create      app/views/comments/show.json.jbuilder
      invoke  assets
      invoke    coffee
      create      app/assets/javascripts/comments.js.coffee
      invoke    scss
```

Fallbacks allow your generators to have a single responsibility, increasing code reuse and reducing the amount of duplication.

8 Application Templates

Now that you've seen how generators can be used *inside* an application, did you know they can also be used to *generate* applications too? This kind of generator is referred as a "template". This is a brief overview of the Templates API. For detailed documentation see the Rails Application Templates guide.

```
gem "rspec-rails", group: "test"
gem "cucumber-rails", group: "test"

if yes?("Would you like to install Devise?")
  gem "devise"
  generate "devise:install"
  model_name = ask("What would you like the user model to be called?
[user]")
  model_name = "user" if model_name.blank?
  generate "devise", model_name
end
```

In the above template we specify that the application relies on the `rspec-rails` and `cucumber-rails` gem so these two will be added to the `test` group in the `Gemfile`. Then we pose a question to the user about whether or not they would like to install Devise. If the user replies "y" or "yes" to this question, then the template will add Devise to the `Gemfile` outside of any group and then runs the `devise:install` generator. This template then takes the users input and runs the `devise` generator, with the user's answer from the last question being passed to this generator.

Imagine that this template was in a file called `template.rb`. We can use it to modify the outcome of the `rails new` command by using the `-m` option and passing in the filename:

```
$ rails new thud -m template.rb
```

This command will generate the `Thud` application, and then apply the template to the generated output.

Templates don't have to be stored on the local system, the `-m` option also supports online templates:

```
$ rails new thud -m https://gist.github.com/radar/722911/raw/
```

Whilst the final section of this guide doesn't cover how to generate the most awesome template known to man, it will take you through the methods available at your disposal so that you can develop it yourself. These same methods are also available for generators.

9 Generator methods

The following are methods available for both generators and templates for Rails.

Methods provided by Thor are not covered this guide and can be found in Thor's documentation

9.1 `gem`

Specifies a gem dependency of the application.